

GRUPO DUPLA · KLICKA

Klicka Prospect

PRD — Camada SaaS & Monetização

Versão 0.1 — rascunho pra aprovação

Autor Natan

Base Next 16 · React 19 · Tailwind v4

1. Objetivo

O motor de extração já funciona (Google Maps + Mercado Livre → enriquecimento → CSV). Falta a **carroceria comercial**: alguém criar conta, usar dentro de um limite, bater no teto e pagar pra liberar mais.

Meta: sair de "ferramenta interna Natan+irmão" pra "**produto que estranho entra, testa e assina sozinho**". Referência de mercado: Faro Leads (faroleads.site) — mesmo motor, empacotado e monetizado.

2. O que já existe (não refazer)

- Busca Google Places (Text Search + Place Details) com autocomplete de localização
- Busca Mercado Livre (OAuth completo: authorize / callback / status)
- Enriquecimento: raspa email, WhatsApp, Instagram do site (cheerio + regex)
- Tabela de resultados com filtro (com site / sem site), seleção, chips de nicho
- Export CSV (completo + só contatos) via papaparse
- UI dark, roxo Klicka, single-page com 3 estados (search → results → enriched)

3. O que falta (escopo deste PRD)

1. **Auth / contas** — Clerk (mesma stack do Klicka Studio)
2. **Banco de dados** — Supabase Postgres + Drizzle (mesma stack do Painel Arena)
3. **Medição de uso** — contar buscas / leads / exports por usuário por mês
4. **Planos + trava** — Free / Pro / Business, limites aplicados no servidor
5. **Cobrança** — gateway plugável (decisão do gateway fica pra Fase 3)
6. **Landing page** — página de venda pública, preços, CTA de cadastro

4. Modelo de negócio

4.1 Métrica de valor e de custo

- **Valor pro cliente:** leads com contato (telefone/WhatsApp/email) exportados.
- **Custo pra nós:** a **Google Places API** é o único custo variável relevante (cada busca = Text Search + Place Details por resultado). Mercado Livre e scraping são de graça (só CPU).

A sacada: o limite de **buscas/mês protege a margem**, e o **gate de export protege o valor**. O "60 por busca" deixa de ser limitação técnica e vira **degrau de plano**.

4.2 Planos (rascunho — números 100% ajustáveis)

	Free	Pro	Business
Preço/mês	R\$ 0	R\$ 67	R\$ 147
Buscas / mês	5	100	400
Resultados por busca	20	60	60
Enriquecer contatos	10 leads (total)	✅ ilimitado	✅ ilimitado
Fontes	Google Maps	Maps + Mercado Livre	Maps + Mercado Livre
Export CSV	❌ (só vê na tela)	✅	✅
Histórico de buscas	❌	30 dias	ilimitado
Suporte	—	e-mail	prioritário

Lógica dos limites:

- **Free** existe pra provar valor e bater no teto rápido: vê os leads na tela, mas não baixa. Sente a dor → assina.
- **Pro** é o plano-âncora (o "certo" pra maioria).
- **Business** é pra quem roda outbound em volume (agências, afiliados com operação).

4.3 Trial

- **Opção A (recomendada):** Free é o próprio trial — sem cartão, sem prazo, trava por limite.
- **Opção B:** 7 dias de Pro grátis com cartão. Decidir junto com o gateway na Fase 3.

5. Arquitetura

5.1 Rotas

Públicas (sem login): `/` landing page · `/precos` · `/entrar` · `/cadastrar` · `/termos` · `/privacidade`

Protegidas (exigem login): `/app` (a ferramenta atual, movida do `/`) · `/conta` (uso do mês, plano, upgrade, assinatura)

5.2 Camadas novas

```
middleware.ts      Clerk: protege /app e /conta, libera público
lib/plans.ts       definição dos planos e limites (fonte da verdade)
lib/db/            Drizzle schema + client (Supabase Postgres)
lib/usage.ts       getUsage / incrementUsage / enforceLimit
lib/billing/       abstração de gateway (interface) – Fase 3
app/api/webhooks/billing webhook do gateway – Fase 3
```

Os API routes atuais (`/api/search`, `/api/ml-search`, `/api/enrich`, `/api/ml-enrich`) ganham 2 linhas no topo: **(1) checa auth, (2) checa limite / incrementa uso**. Se estourou, devolve `402 Payment Required` com o plano sugerido, e o front mostra o modal de upgrade.

5.3 Schema do banco (Drizzle / Postgres)

```
users
  id          text PK      -- Clerk user id
  email       text
  plan        text         -- 'free' | 'pro' | 'business' (default 'free')
  plan_status text         -- 'active' | 'past_due' | 'canceled'
  period_end  timestampz   -- fim do ciclo pago atual (null no free)
  gateway_customer_id text -- id do cliente no gateway (Fase 3)
  gateway_subscription_id text -- id da assinatura (Fase 3)
  created_at  timestampz default now()

usage_counters -- 1 linha por usuário por mês
  user_id      text FK -> users.id
  period       text     -- 'YYYY-MM'
  searches_count int default 0
  leads_extracted int default 0
  exports_count int default 0
  PRIMARY KEY (user_id, period)

searches -- histórico (feature de plano pago)
  id          uuid PK
  user_id     text FK
  source      text     -- 'places' | 'mercadolivre'
  query       text
  location    text
  result_count int
  created_at  timestampz default now()
```

```
billing_events -- log de webhook do gateway (Fase 3)
  id, user_id, type, payload jsonb, created_at
```

Persistir os leads em si (tabela `leads`) fica pra depois — não é necessário pra monetizar. Histórico guarda a *busca*, não cada linha.

6. Regras de medição (o coração)

- Contador vive em `usage_counters`, chave `(user_id, 'YYYY-MM')`. Reseta naturalmente virando o mês (nova linha).
- Busca:** antes de chamar Places/ML, `enforceLimit('searches')`. Se atingiu o limite do plano → 402. Senão incrementa e segue.
- Resultados por busca:** corta a lista no limite do plano (Free 20, Pro/Business 60) **no servidor**, não no client.
- Enriquecer (Free):** conta os enriquecidos; Free trava em 10 no total.
- Export:** Free não baixa (botão vira "Assine o Pro pra exportar"). Pago só incrementa métrica.
- Tudo validado **no servidor**. O client só reflete — nunca é a fonte da trava.

7. Fases de execução

Fase	Entrega	Depende de você
1. Fundação	Clerk (login/cadastro) + Supabase + Drizzle + schema criado + ferramenta movida pra <code>/app</code> + middleware	Chaves do Clerk + connection string do Supabase
2. Medição + trava	<code>lib/plans.ts</code> , <code>lib/usage.ts</code> , limites nos 4 API routes, página <code>/conta</code> com uso do mês, modal de upgrade	Só validar os números dos planos
3. Cobrança	Gateway escolhido, checkout, webhook ativando/desativando plano, portal de assinatura	Conta + chaves do gateway
4. Landing page	LP pública nível Faro: hero, como funciona, features, prova, preços, FAQ, CTA → cadastro	Claims que quer bater (ou eu escrevo e você revisa)

Cada fase é deployável sozinha. Ordem recomendada: **1 → 2 → 4 → 3** (vende/valida com LP + login antes de ligar a cobrança), ou **1 → 2 → 3 → 4** se quiser faturar antes.

8. O que preciso de você pra rodar a Fase 1

- Clerk:** reusar o app do Klicka Studio ou criar um novo? (preciso das chaves `NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY` e `CLERK_SECRET_KEY`)

2. **Supabase:** projeto novo pro Prospect ou reusar um existente? (preciso da `DATABASE_URL`)
3. Confirmar/ajustar os **números dos planos** da seção 4.2.

9. Fora de escopo (por ora)

- App nativo Windows (decidido: fica web/SaaS)
 - Persistir cada lead em DB (só guarda histórico da busca)
 - Equipes/multi-seat (Clerk Orgs) — dá pra ligar depois se virar B2B de agência
 - Dedupe automático, integração direta com Instantly/Smartlead (backlog do PRD original)
-

Klicka Prospect · PRD Camada SaaS/Monetização v0.1 · Documento interno Grupo Dupla